

Cuestionario clases 1 y 2

1. Mencione al menos 3 ejemplos donde pueda encontrarse concurrencia

(Voy a asumir que se refiere a ejemplos de aplicaciones en la vida real)

Encontramos concurrencia en los juegos multijugador online, en los sitios de streaming, y en las bases de datos de los bancos

2. Escriba una definición de concurrencia. Diferencie procesamiento secuencial, concurrente y paralelo.

Concurrencia es una propiedad que tienen los programas informáticos de realizar mas de una tarea a la vez. La diferencia entre el procesamiento secuencial, concurrente, y paralelo, estriba en la cantidad de tareas que se realizan en simultáneo, y en la ejecución de instrucciones que ocurren en un instante dado.

- En el caso del **procesamiento secuencial**, el programa realiza una sola tarea, y en cada instante se está ejecutando una sola instrucción.
- En el caso del **procesamiento concurrente**, el programa puede realizar mas de una tarea a la vez, como ejemplo tenemos al algoritmo de planificación de procesos Round Robin, pero en cada instante sigue ejecutando una sola instrucción.
- En el caso del **procesamiento paralelo**, el programa puede realizar mas de una tarea a la vez y ejecutar mas de una instrucción a la vez. Requiere que la computadora use mas de un procesador por lo menos.

3. Describa el concepto de deadlock y qué condiciones deben darse para que ocurra

Un **deadlock** es una situación en la que un programa se queda detenido hasta que se fuerze su terminación debido a que dos, o mas, procesos del programa están pidiendo recursos que otro tiene.

Para que ocurra **deadlock**, se mencionan las cuatro siguientes condiciones:

- **Recursos reusables serialmente**
- **Adquisición incremental** → Los procesos mantienen los recursos que poseen mientras esperan adquirir otros
- **No-preemption** (no expropiación del recurso) → Los recursos se liberan voluntariamente
- **Espera cíclica** → Existe una cadena circular de procesos en la que cada uno se queda esperando por el recurso que tiene el anterior

4. Defina inanición. Ejemplifique

La **inanición** es el estado en el que un proceso no recibe instantes para ejecutarse. El proceso no puede ejecutarse debido a que no tiene acceso a los recursos o las condiciones necesarias activadas.

Por ejemplo, si abrimos una página y hacemos que los usuarios que entren deban ingresar su edad, así podemos darle prioridad a aquellas personas menores o ancianas, podría pasar que vayan llegando niños y ancianos para siempre, y que entonces las personas de mediana edad nunca lleguen a entrar.

5. ¿Qué entiende por no determinismo? ¿Cómo se aplica este concepto a la ejecución concurrente?

El **no determinismo** es una propiedad de un programa que, dado una entrada, o unas condiciones iniciales, el programa termina con uno o mas outputs o condiciones finales diferentes.

Como los programas concurrentes tienen varios procesos ejecutándose en simultáneo, y generalmente estos procesos suelen compartir algunos de los recursos, esto puede provocar que la invocación del mismo programa no arroje la misma salida si se lo invoca dos veces.

6. Defina comunicación. Explique los mecanismos de comunicación que conozca.

La comunicación es un proceso que ocurre entre procesos (informáticos) de un programa por el cual estos envían y reciben datos de manera organizada.

Los mecanismos de comunicación que conozco son los siguientes:

- **Variables compartidas** → Los procesos utilizan una misma variable global. A veces se tienen que turnar para leerla y modificarla, debido a que de otro modo puede haber **interferencia**.
- **Pasaje de mensajes** → Los procesos utilizan canales para pasarse información y coordinarse.

7. a. Defina sincronización. Explique los mecanismos de sincronización que conozca.

La sincronización es la posesión de información con la cual dos o mas procesos (informáticos) se pueden coordinar.

- **Sincronización por exclusión mutua** → Asegurar que sólo un proceso tenga acceso a un recurso compartido en un instante de tiempo
- **Sincronización por condicion** → Permite bloquear la ejecución de un proceso hasta que se cumpla una condición dada

7. b. ¿En un programa concurrente pueden estar presentes más de un mecanismo de sincronización? En caso afirmativo, ejemplifique

Si, es totalmente posible. Por ejemplo, piénsese en un programa donde haya una barrera que sincronice la entrada de 50 alumnos (*sincronización por condición*), y luego estos alumnos tengan que ir hasta la mesa de la profesora para entregarle su tarea, de a uno a la vez (*sincronización por exclusión mutua*)

8. ¿Qué significa el problema de "interferencia" en programación concurrente? ¿Cómo puede evitarse?

El problema de "interferencia" ocurre cuando el valor que tiene un recurso compartido no es el que se esperaba que tuviera un proceso, debido a que otro proceso lo manipuló antes. Se puede evitar coordinando o sincronizando el acceso a un recurso compartido, por ejemplo, por medio de exclusión mutua.

9. ¿En qué consiste la propiedad de "A lo sumo una vez" y qué efecto tiene sobre las sentencias de un programa concurrente? De ejemplos de sentencias que cumplan y de sentencias que no cumplan con ASV.

Tenía algo que ver con que un recurso compartido referenciara, a lo sumo, a otro recurso compartido y nada mas. Esta propiedad garantiza que las sentencias, o grupos de sentencias si hay atomicidad de grano grueso, se puedan atomizar para aplicar exclusión mutua y así proteger los recursos.

$x = x + 1;$ → **Cumple ASV**

$x = x + 1; y = x + 1$ → **Cumple ASV (hay solo 1 referencia crítica)**

$x = y + 1; y = x + 1;$ → **NO cumple ASV**

10.

Dado el siguiente programa concurrente:

```
x = 2; y = 4; z = 3;
```

```
co
```

```
  x = y - z // z = x * 2 // y = y - 1
```

```
oc
```

a) ¿Cuáles de las asignaciones dentro de la sentencia *co* cumplen con ASV?. Justifique claramente.

b) Indique los resultados posibles de la ejecución

Nota 1: las instrucciones NO SON atómicas.

Nota 2: no es necesario que liste TODOS los resultados, pero si los que sean representativos de las diferentes situaciones que pueden darse.

a. Las últimas dos sentencias cumplen con ASV. La primera no cumple con la propiedad ASV debido a que se está haciendo mas de una referencia crítica en la asignación, mientras que en las otras no.

b. (1.1) $(y - z)$ // (1.2) $(x = y - z)$
(2.1) $(x * 2)$ // (2.2) $(z = x * 2)$
(3) $(y = y - 1)$

(1.1) (1.2) (2.1) (2.2) (3)

$x = 1$

$y = 3$

$z = 2$

(2.1) (2.2) (1.1) (1.2) (3)

$x = 0$

$y = 4$

$z = 4$

(2.1) (2.2) (3) (1.1) (1.2)

x = -1

y = 3

z = 4

(3) (2.1) (2.2) (1.1) (1.2)

x = -1

y = 3

z = 4

(1.1) (2.1) (3) (1.2) (2.2)

x = 1

y = 3

z = 4

11. Defina acciones atómicas condicionales e incondicionales. Ejemplifique

Las acciones atómicas condicionales son las acciones que se ejecutan de forma atómica cuando se cumple cierta condición. Las acciones atómicas incondicionales son las que se ejecutan siempre.

Un ejemplo de acción atómica condicional es el decremento de una variable contador si el valor de esta variable es mayor a 0

12. Defina propiedad de seguridad y propiedad de vida.

- **Propiedad de seguridad** → Propiedad que garantiza que un programa concurrente, en cualquiera de sus historias de ejecución, va a tener procesos con estados consistentes. Un ejemplo sería la exclusión mutua
- **Propiedad de vida** → Propiedad que garantiza que en un programa concurrente los procesos progresan (es decir, no hay inanición), y no hay deadlocks (el programa nunca se va a colgar)

13. ¿Qué es una política de scheduling? Relacione con fairness. ¿Qué tipos de fairness conoce? ¿Por qué las propiedades de vida dependen de la política de scheduling?

Una política de scheduling es una política que determina el orden y la forma en la que las acciones atómicas de los procesos serán elegidos para ejecutarse.

El **fairness** es una propiedad que trata de garantizar que los procesos tengan posibilidades de avanzar, sin importar lo que hagan los demás procesos (por lo tanto sería de vida)

La **política de scheduling** y el **fairness** están directamente relacionados porque el mecanismo que determina qué proceso será elegido garantizará mas o menos las posibilidades que tienen los procesos de avanzar.

Conozco estos dos tipos:

- **Fairness incondicional** → Toda **acción atómica incondicional** que es elegible se ejecuta en algún instante del programa
- **Fairness débil** → Es incondicionalmente fair, y toda **acción atómica condicional** que se vuelve elegible (su condición se cumple) se ejecuta en algún momento del programa, asumiendo que su condición se cumple y permanece verdadera hasta que sea vista por el proceso que ejecuta la **acción atómica condicional** → Las acciones atómicas condicionales cuyas condiciones duraron mas tiempo como verdaderas, tienen mas probabilidad de ejecutarse
- **Fairness fuerte** → Es incondicionalmente fair, y toda **acción atómica condicional** que se vuelve elegible se ejecuta en algún momento del programa porque su condición se vuelve true con "infinita frecuencia" → Las acciones atómicas condicionales cuyas condiciones se volvieron verdaderas mas veces, tienen mas probabilidades de ejecutarse

14.

Dado el siguiente programa concurrente, indique cuál es la respuesta correcta (justifique claramente)

```
int a = 1, b = 0;
```

```
co ⟨await (b = 1) a = 0⟩ // while (a = 1) { b = 1; b = 0; } oc
```

a) Siempre termina

b) Nunca termina

c) Puede terminar o no

- $C \rightarrow$ **Puede terminar o no**, dependiendo de si ocurre lo siguiente:
 - (1) $\langle \text{await } (b=1) \text{ a} = 0 \rangle$
 - (2.1) $\text{while } (a = 1) \{ b = 1 \}$
 - (2.2) $\{ b = 0 \}$
 1. (2.1)
 2. (1)
 3. (2.2) $\rightarrow$ Para este punto, ambas variables valdrán 0, por lo que ninguna sentencia se ejecutará, y el programa terminará

15. ¿Qué propiedades se deben garantizar en la administración de una sección crítica en procesos concurrentes? ¿Cuáles de ellas son propiedades de seguridad y cuáles de vida? En el caso de las propiedades de seguridad, ¿cuál es en cada caso el estado "malo" que se debe evitar? Para cada propiedad explique una solución que no la cumpla

Deben garantizarse las siguientes propiedades en la administración de una sección crítica:

- **Exclusión mutua** $\rightarrow$ Propiedad de seguridad
- **Ausencia de Deadlock** $\rightarrow$ ¿Propiedad de seguridad o de vida? $\rightarrow$ Se puede justificar como cualquiera de las dos me dijo el profesor
- **Ausencia de Demora Innecesaria** $\rightarrow$ Propiedad de vida
- **Eventual Entrada** $\rightarrow$ Propiedad de vida

16. Resuelva el problema de acceso a sección crítica para N procesos usando un proceso coordinador. En este caso, cuando un proceso $SC[i]$ quiere entrar a su sección crítica le avisa al coordinador, y espera a que éste le otorgue permiso. Al terminar de ejecutar su sección crítica, el proceso $SC[i]$ le avisa al coordinador. Desarrolle una solución de grano fino usando únicamente variables compartidas (ni semáforos ni monitores).

```
int arribo[1:n] = ([n] 0), continuar[1:n] = ([n] 0);
```

```

process Worker[i=1 to n]
{
    while (true) {
        código para implementar la tarea i;
        // Quedarse esperando mientras el Coordinador le otorga el acceso
        while (continuar[i] == 0) skip;
        continuar[i] = 0;
        termino[i] = 1;
    }
}

process Coordinador
{
    while (true) {
        for [i = 1 to n] {
            continuar[i] = 1;
            while (termino[i] == 0) skip;
            termino[i] = 0;
        }
    }
}

```

17. ¿Qué mejoras introducen los algoritmos Tie-breaker, Ticket o Bakery en relación a las soluciones de tipo spin- locks?

- **Tie-breaker** → A diferencia del spin lock, permite controlar el orden de los procesos demorados (una variable adicional que rompe empates, indicando qué proceso fue el último en comenzar dicha entrada "tie-breaker")
- **Ticket** → A diferencia del spin lock, introduce un orden de ejecución para los procesos (asignándole a cada proceso un turno, y encargándose de incrementar la variable turno para el próximo proceso, como si se tratara de sacar un ticket en una tienda). En relación a la solución anterior, es menos costosa en tiempo y compleja
- **Bakery** → A diferencia del spin lock, introduce un orden en la ejecución de los procesos como el algoritmo Ticket, y a diferencia de este último, evita el problema de que la solución no sea **fair** por no poder buscar e incrementar el

número del turno de manera atómica. Los procesos se chequean entre ellos y no contra un global.

18. Analice las soluciones para las barreras de sincronización desde el punto de vista de la complejidad de la programación y de la performance

- La **solución mas simple en términos de complejidad** es la del **contador compartido**, que requiere prácticamente dos sentencias y una variable global. El problema es cuando se tenga que resetear la barrera para que pueda servir como barrera la próxima vez (¿se realiza dentro del mismo proceso?), lo que hace que no sean tan óptima en términos de performance.
- La solución de **Flags y Coordinadores** permite que **la barrera sirva para cada iteración**, pero introduce complejidad (un proceso de mas y dos arreglos, lo que requiere por lo menos un procesador mas) y pérdida de performance (hay que esperar a que el coordinador haya iterado sobre el primer arreglo para que cada proceso pueda pasar)
- La solución de **arboles** combina las acciones de Workers y Coordinadores, haciendo que cada Worker sea también Coordinador y mande de arriba para abajo las señales de arribo. Es mas compleja, pero en términos de performance mas eficiente para n grande
- La **barrera simétrica** funciona para sincronizar dos procesos. Es simple de implementar y gasta poca performance, sin embargo por si sola solo puede sincronizar pares de procesos nada mas
- La **butterfly barrier** es una forma de combinar pares de barreras simples para dos procesos. Cada worker sincroniza con un worker distinto en cada etapa, en la etapa S, un worker sincroniza con otro a distancia $2^S + 1$, y cuando cada Worker pasó $\log_2 n$ etapas, todos pueden seguir. Esta es **la mejor en términos de performance**

19. Explique gráficamente cómo funciona una butterfly barrier para 8 procesos usando variables compartidas

